

N integers, you are asked to find out whether there are three numbers a , b and c in the set of N numbers whose sum is equal to zero. The simplest algorithm can consider each triple of the numbers at a time, and check whether any triple satisfies the criteria of $a+b+c=0$. The following is the pseudo-code for this:

```
for i = 1 to N-2
  for j = i+1 to N-1
    for k = j+1 to N
      if (A[i] + A[j] + A[k] == 0)
        Return the triple (A[i], A[j], A[k]);
    if no such triple found,
      return false;
```

What is the time complexity of the above algorithm? We can see that since there are three *for* loops running over N , hence the complexity is $O(N^3)$. Can we do better than this? Can we come up with a $O(n^2)$ algorithm for this problem?

$O(N^2)$ solution to the 3-SUM problem

We saw that in the above approach, we picked up triples blindly and hence ended up with $O(N^3)$ complexity. How can we pick up triples more cleverly? Let us first sort the set of input numbers. Now we compare $A[1]$, $A[2]$ and $A[N]$. If the sum is zero, return the triple. If the sum is greater than zero, we compare $A[1]$, $A[2]$ and $A[N-1]$. And if the sum is less than zero, we compare $A[1]$, $A[3]$ and $A[N]$. Basically, since the numbers are sorted in increasing order, if the sum is above zero, we need to reduce the sum by using a smaller operand. If the sum is negative, we need to increase the value of the operand to make the sum zero. Given below is the pseudo code for this problem:

```
bool is_3_sum(array A[], N)
{
  for i = 1 to N-2
  {
    j = i+1; k = N
    while (k > j)
    {
      if (A[i] + A[j] + A[k] == 0)
        Return true;
      else if (A[i] + A[j] + A[k] > 0)
        k = k - 1;
      else
        j = j + 1 //case when the sum is less than zero
    }
    return false;
  }
}
```

What is the complexity of the above algorithm? In the inner loop, during each iteration, we eliminate one element of the array from consideration; hence,

the inner loop runs for a maximum of N iterations and the outer loop runs for a maximum of $(N-2)$ iterations. Hence, the overall complexity of the algorithm is $O(N^2)$. Now the interesting question is to determine whether we can come up with a sub-quadratic algorithm for this problem?

Theoretical significance of the 3-SUM problem

Interestingly, all the research so far has not been able to come up with a sub-quadratic algorithm. The best measure of the complexity obtained so far has only been $O(N^2)$. Hence, it is widely believed that there is no sub-quadratic algorithm for this problem. However, this lower bound has not been theoretically established for all general models of computation, so the problem of finding a sub-quadratic solution to the 3-SUM problem still remains an open question in the field of algorithms.

The 3-SUM problem is interesting not just because we have not been able to find a sub-quadratic algorithm, but because many of the computation geometry problems can be reduced to an instance of the 3-SUM problem. There are problems like 3-point collinearity, where, given a set of N points, we need to decide whether any of the three points are collinear. A similar problem is in finding the minimum area triangle formed by three points from a given set of N points.

It can be shown that both these problems can be reduced to a 3-SUM problem in sub-quadratic time. Hence, if a sub-quadratic solution is found for the 3-SUM problem, both 3-point collinearity and the minimum area triangle also can be solved in sub-quadratic time. However, till date, no sub-quadratic solution has been determined for any of these problems.

This month's takeaway problem

For this month's takeaway problem, let us consider a variant of the 3-SUM problem. You are given three sets of numbers A , B and C , each containing N numbers. Can you come up with an algorithm to determine whether there is a triple— $a \in A$, $b \in B$ and $c \in C$ —such that $a + b = c$. It is quite easy to come up with a $O(N^2 \log N)$ algorithm. But you need to come up with a $O(N^2)$ algorithm for this. Here's a hint: use a variant of the 3-SUM algorithm to solve this problem.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me. Feel free to send your solutions and feedback at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

Sandya Mannarswamy

The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.